

Table des matières

Exercices 10 — Index, plans d'exécution et optimisation	1
Questions courtes	1
Questions pratiques	2
Diagnostics	3
Questions à développer	5
Mises en situation	5

Exercices 10 — Index, plans d'exécution et optimisation

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL et lire des plans ; les diagnostics vous montrent du code ou des plans qui clochent ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez trois heures et quart pour y répondre, le corrigé non compris.

Les repères de terrain. Ce module n'a qu'un laboratoire, sur PostgreSQL. Un énoncé marqué [OLTP] se traite donc dans PostgreSQL sur Adventureworks, ou sur le schéma labo10 que vous avez construit. Les rares énoncés marqués [OLAP] se traitent dans DuckDB, sur l'entrepôt. Sans marque, la question ne demande pas de moteur.

Le corrigé est dans `Corrige_10.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Pourquoi mesure-t-on deux ou trois fois avant de conclure qu'une requête est plus rapide ?
2. Un index B-tree est une structure triée. Citez trois choses que ce tri lui permet de faire.
3. Nommez les trois coûts d'un index, et dites lequel est payé même quand personne ne lit la table.
4. EXPLAIN et EXPLAIN ANALYZE. Quelle est la différence, et quelle précaution prendre avec le second sur un UPDATE ?
5. Dans quel sens se lit un plan d'exécution ?
6. Quel est le signal de diagnostic le plus utile d'un plan, et que veut dire un écart d'un facteur cent ?

7. Sur une boucle imbriquée, pourquoi ne faut-il pas lire le temps du nœud interne tel quel ?
 8. Donnez trois nœuds de jointure et dites dans quel cas chacun est le bon choix.
 9. Définissez la sargabilité en une phrase, sans employer le mot anglais dont elle vient.
 10. Réécrivez `WHERE EXTRACT(YEAR FROM orderdate) = 2023` de façon sargable, et dites ce que la réécriture a déplacé.
 11. `WHERE nom LIKE '%150'` ne peut employer aucun index B-tree. Pourquoi ?
 12. Un index B-tree ordinaire sur du texte ne sert pas toujours `LIKE 'préfixe%'`. Donnez la cause et la parade.
 13. Un développeur corrige une erreur de type en écrivant `WHERE cle::int = 29825`. Qu'a-t-il cassé, et qu'aurait-il fallu écrire ?
 14. Énoncez la règle du préfixe le plus à gauche, avec une analogie, puis dites ce que PostgreSQL 18 fait d'un filtre qui saute la première colonne de l'index.
 15. Dans un index composite, où placez-vous la colonne filtrée par égalité et où placez-vous la plage ? Justifiez.
 16. Qu'est-ce qu'un index couvrant, et à quoi sert le mot-clé `INCLUDE` ?
 17. Citez les quatre situations où un index nuit plus qu'il n'aide, et donnez la parade pour celle de la colonne peu sélective.
 18. Comment sait-on qu'un index de production ne sert jamais ?
 19. Sur quoi le planificateur s'appuie-t-il pour choisir un plan, et dans quelles trois situations cette information le trompe-t-elle ?
 1. L'entrepôt n'a pas d'index utile. Expliquez ce qui remplace l'index, dites ce qu'il faut regarder pour savoir si cela a fonctionné, et pourquoi ce diagnostic est plus difficile que sur le transactionnel.
-

Questions pratiques

- P1. [OLTP] Sur le schéma `labo10`, écrivez la requête qui trouve les dix commandes du client 29825 en 2023, puis mesurez-la sans index. Créez ensuite l'index qui la sert, remesurez, et donnez le rapport entre les deux temps.
- P2. [OLTP] Écrivez trois requêtes qui renvoient exactement le même résultat, dont une seule emploie un index existant. Prouvez-le par les plans.
- P3. [OLTP] Concevez l'index le mieux adapté à cette requête, et justifiez l'ordre des colonnes :

```
SELECT ligne_id, linetotal
FROM   labo10.grosse
WHERE  customerid = 29825 AND jour >= '2023-01-01'
ORDER BY jour;
```

- P4. [OLTP] Faites de l'index de P3 un index couvrant. Vérifiez que le plan devient un parcours d'index seul, et dites ce qui se passerait si vous ajoutiez produit au SELECT.
- P5. [OLTP] Écrivez la requête qui liste, pour le schéma labo10, chaque index avec sa taille et son nombre d'utilisations, du moins utilisé au plus utilisé.
- P6. [OLTP] Créez un index partiel qui ne couvre que les lignes de plus de 1 000 \$, puis écrivez une requête qu'il sert et une requête très proche qu'il ne sert pas. Expliquez la frontière. (avancé)
- P7. [OLAP] Sur l'entrepôt, écrivez deux requêtes équivalentes sur une table rangée par date, dont une seule permet l'élagage. Mesurez les deux et dites ce que vous regardez pour le prouver.
- P8. [OLTP] Une table de 40 millions de lignes vient d'être chargée. Écrivez la séquence de commandes que vous lancez avant de mesurer quoi que ce soit, et justifiez chaque commande en une phrase. (avancé)
-

Diagnostics

E1. Sur le schéma labo10, avec l'index idx_grosse_produit de la partie 2, cette requête joint les lignes de ventes à leurs en-têtes de commande :

```
SELECT SUM(h.freight)
FROM   labo10.grosse AS g
JOIN   sales.salesorderheader AS h ON h.salesorderid = g.salesorderid
WHERE  g.productid = 870 AND g.produit = 'Water Bottle - 30 oz.' AND g.unitprice = 4.99;
```

PostgreSQL 18 rend ce plan, lancé par EXPLAIN ANALYZE :

```
Aggregate  (cost=16058.26..16058.27 rows=1 width=32) (actual time=77.817..77.817 rows=1.00 loops=1)
  Buffers: shared hit=131162 read=65
    -> Nested Loop  (cost=529.66..16057.91 rows=139 width=6) (actual time=1.589..72.688 rows=42440.00 loops=1)
      Buffers: shared hit=131162 read=65
        -> Bitmap Heap Scan on grosse g  (cost=529.38..15195.51 rows=139 width=4) (actual time=1.566..15.618 rows=42440.00
loops=1)
          Recheck Cond: (productid = 870)
          Filter: (((produit)::text = 'Water Bottle - 30 oz.'::text) AND (unitprice = 4.99))
          Rows Removed by Filter: 4440
          Heap Blocks: exact=3865
          Buffers: shared hit=3907
        -> Bitmap Index Scan on idx_grosse_produit  (cost=0.00..529.34 rows=48122 width=0) (actual time=1.136..1.136
rows=46880.00 loops=1)
          Index Cond: (productid = 870)
          Index Searches: 1
          Buffers: shared hit=42
```

```

-> Index Scan using "PK_SalesOrderHeader_SalesOrderID" on salesorderheader h (cost=0.29..6.20 rows=1 width=10) (actual
time=0.001..0.001 rows=1.00 loops=42440)
    Index Cond: (salesorderid = g.salesorderid)
    Index Searches: 42440
    Buffers: shared hit=127255 read=65
Execution Time: 77.908 ms

```

Le moteur a choisi une boucle imbriquée, et la requête met plus de deux fois le temps qu'elle prend en Hash Join, en touchant vingt-huit fois plus de pages. Expliquez ce qui l'a conduit à ce choix, et dites ce que vous feriez.

E2. Un développeur signale que son index ne sert pas :

```

CREATE INDEX idx_jour ON commande (jour);
SELECT COUNT(*) FROM commande WHERE jour + INTERVAL '1 day' > now();

```

Expliquez et corrigez.

E3. Sur labo10.grosse, un index a été créé ainsi, pour servir deux requêtes, et c'est le seul index de la table :

```

CREATE INDEX idx_jour_client ON labo10.grosse (jour, customerid);
-- Requête A
SELECT COUNT(*) FROM labo10.grosse WHERE customerid = 29825;
-- Requête B
SELECT COUNT(*) FROM labo10.grosse WHERE jour >= '2023-01-01' AND customerid = 29825;

```

Sous PostgreSQL 18, les deux plans emploient l'index, et son auteur en conclut qu'il sert bien les deux requêtes. Les plans affichent pourtant Index Searches : 414 pour A et Index Searches : 398 pour B. Dites ce que ces nombres comptent, pourquoi ils sont si élevés, si l'auteur a raison, et quel index vous créeriez à la place.

E4. Sur le schéma labo10, un index sur grosse (salesorderid) sert cette jointure des commandes de juin 2025 à leurs lignes :

```

CREATE INDEX idx_grosse_commande ON labo10.grosse (salesorderid);

SELECT h.salesorderid, g.linetotal
FROM sales.salesorderheader AS h
JOIN labo10.grosse AS g ON g.salesorderid = h.salesorderid
WHERE h.orderdate >= '2025-06-01';

```

Son plan, par EXPLAIN ANALYZE :

```

Nested Loop (cost=0.43..9328.67 rows=35896 width=11) (actual time=2.065..7.596 rows=20470.00 loops=1)
  Buffers: shared hit=4558
  -> Seq Scan on salesorderheader h (cost=0.00..1115.31 rows=931 width=4) (actual time=2.056..2.164 rows=905.00 loops=1)
    Filter: (orderdate >= '2025-06-01 00:00:00'::timestamp without time zone)

```

```
Rows Removed by Filter: 30560
Buffers: shared hit=722
-> Index Scan using idx_grosse_commande on grosse g (cost=0.43..8.20 rows=62 width=11) (actual time=0.001..0.003 rows=22.62
loops=905)
    Index Cond: (salesorderid = h.salesorderid)
    Index Searches: 905
    Buffers: shared hit=3836
Execution Time: 8.143 ms
```

Un collègue conclut que le parcours d'index ne pèse rien, puisqu'il prend 0,003 milliseconde, et qu'il faut chercher le temps ailleurs. Corrigez-le, chiffres à l'appui, puis dites si la boucle imbriquée était le bon choix.

E5. Une équipe a ajouté onze index sur une table d'événements alimentée par un flux temps réel. Les lectures sont un rapport quotidien. Les insertions ont ralenti d'un facteur quatre. Décrivez la démarche que vous suivriez, dans l'ordre, et dites quelle requête vous écririez en premier. (avancé)

E6. Sur l'entrepôt, une requête filtrée sur une seule journée met le même temps qu'un balayage complet. Le développeur en déduit qu'il faut réécrire toute la table dans un autre ordre. Donnez deux causes plus probables à vérifier avant, et dites comment vous les vérifiez.

Questions à développer

A1. La théorie compare un index à un abonnement plutôt qu'à un achat. Développez cette image en un paragraphe, puis dites quelles données vous rassembleriez pour décider de supprimer un index existant en production. Précisez ce que vous feriez avant la suppression pour pouvoir revenir en arrière.

A2. Un architecte propose une règle simple : « on indexe toutes les colonnes qui apparaissent dans un WHERE ». Répondez-lui en un paragraphe, en reconnaissant ce que la règle a de juste, puis proposez une règle de remplacement qui tienne en deux phrases.

A3. Comparez la façon dont chacun des deux terrains rend un filtre rapide. Votre réponse doit expliquer pourquoi l'un demande une structure explicite et l'autre non, dire ce que le second ne dispense pas de faire, et nommer la seule notion du module qui vaut identiquement des deux côtés.

A4. Le module affirme que l'écart entre les lignes estimées et les lignes réelles est le meilleur signal de diagnostic. Expliquez pourquoi cet écart est plus informatif que le temps d'exécution, et décrivez trois causes possibles d'un écart d'un facteur cent, avec le traitement de chacune. (avancé)

Mises en situation

M1. Fonderie Bécancour, le rapport qui s'est mis à ramer. Le rapport de production tourne chaque nuit depuis quatre ans. Il lisait une table de coulées en huit minutes ; depuis six semaines, il en met cinquante, et parfois il n'a pas fini quand l'équipe du matin arrive. Personne n'a modifié le SQL. La table a grossi de trente pour cent sur la période, et une nouvelle application écrit dedans en continu depuis deux mois.

Décrivez votre démarche de diagnostic, dans l'ordre, en nommant à chaque étape la commande ou la vue système que vous consultez. Formulez au moins trois hypothèses compatibles avec les faits de l'énoncé, et dites pour chacune la mesure qui la confirme ou l'écarte. Précisez ce que vous ne feriez pas en premier, et pourquoi.

M2. Grossiste alimentaire Yamaska, l'index qu'on vous demande. Un développeur vous écrit que la page de recherche des commandes est lente, et vous demande d'ajouter un index sur `date_livraison`. La page permet de filtrer par entrepôt, par statut, par plage de dates de livraison et par nom de client partiel, et l'utilisateur remplit rarement plus de deux de ces quatre champs. La table compte 90 millions de lignes et reçoit 400 écritures par seconde.

Répondez au développeur. Votre réponse doit dire ce que vous demandez avant d'agir, expliquer pourquoi un seul index sur `date_livraison` a peu de chances de régler le problème, et proposer une stratégie d'indexation qui tienne compte des quatre champs sans en créer quatre. Traitez explicitement le cas du nom de client partiel, qui est le plus délicat. Dites enfin ce que vous surveilleriez après la mise en place.

M3. Billetterie Matapédia, la migration vers l'entrepôt. Matapédia déplace ses rapports d'une base PostgreSQL vers un entrepôt. L'équipe a passé trois ans à ajuster une trentaine d'index sur la base transactionnelle, et le responsable technique demande la liste des index à recréer dans l'entrepôt pour ne pas perdre les gains.

Répondez-lui. Expliquez ce qui remplace les index, ce qui se transpose et ce qui ne se transpose pas, et ce que devient l'expertise accumulée par l'équipe : dites précisément quelle partie de son travail reste valable et quelle partie devient sans objet. Proposez ensuite le petit nombre de décisions qu'il faut réellement prendre sur l'entrepôt, et dites comment vous les valideriez par la mesure plutôt que par l'intuition. (avancé)